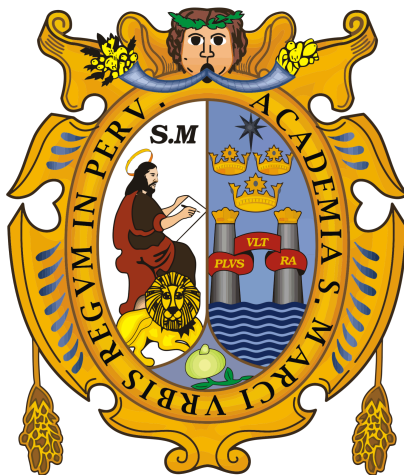


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, DECANA DE AMÉRICA)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



DOCENTE:

Hernan Villafuerte

CURSO:

CIRCUITOS DIGITALES

TEMA:

***“Diseño e Implementación de un Generador de Patrones de Prueba Pseudoaleatorios
(PRBS) Configurable mediante Tabla en ROM y Decodificador”***

INTEGRANTES:

Carrion Rubio Therry

Prieto Portal, Flor Arian

Terrel Gonzales Jhon

Lima-Perú

2026

1. OBJETIVO DEL PROYECTO

Diseñar, codificar en lenguaje de descripción de hardware (HDL) y simular un generador de secuencias pseudoaleatorias (PRBS) cuya "semilla" (valor inicial) es seleccionada por el usuario mediante interruptores. El sistema utiliza un decodificador de direcciones y una memoria ROM para almacenar y cargar las semillas.

2. DISEÑO DETALLADO DEL SISTEMA

La arquitectura del proyecto se divide en tres bloques principales:

2.1. Decodificador (2 a 4): Lee los 2 interruptores de entrada y habilita una de las 4 líneas de salida.

2.2. Memoria ROM (4x8 bits): Almacena 4 semillas iniciales predefinidas. Recibe la señal del decodificador y envía la semilla correspondiente.

2.3. Generador PRBS: Un registro de desplazamiento con retroalimentación lineal (LFSR) que toma la semilla y genera el patrón pseudoaleatorio de 8 bits.

TABLAS DE VERDAD

Tabla 1: Decodificador de Direcciones

Entrada A1	Entrada A0	Salida D3	Salida D2	Salida D1	Salida D0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Tabla 1: Decodificador de Direcciones

Tabla 2: Contenido de la Memoria ROM

Dirección Seleccionada	Semilla Almacenada (Binario)
0	10101011
1	11110000
10	1111
11	11001100

Tabla 2: Contenido de la Memoria ROM

3. MÁQUINA DE ESTADOS FINITOS (FSM) Y DIAGRAMA ASM

Para coordinar el flujo de datos entre los bloques, se diseñó una Unidad de Control basada en una FSM con 4 estados.

Estados de la FSM:

- **IDLE (00):** Estado de reposo. El sistema espera a que la señal **start** sea 1.
- **READ_ROM (01):** El sistema lee la dirección actual y extrae el dato de la ROM.
- **LOAD_PRBS (10)** La semilla extraída de la ROM se carga en el registro del PRBS.
- **RUN (11):** El PRBS comienza a iterar y generar los números pseudoaleatorios. Si **start** vuelve a 0, regresa a IDLE.

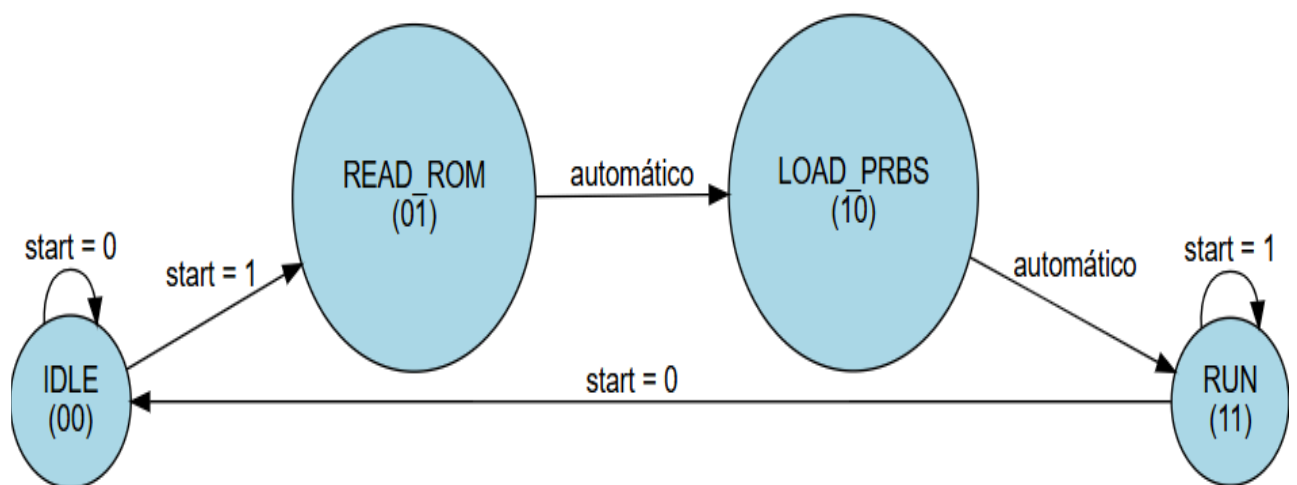


Figura 2: Estados de la FSM

Lógica del Diagrama ASM:

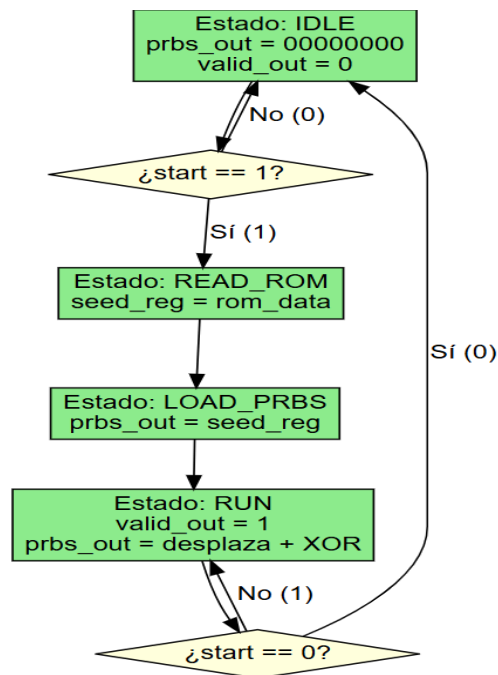
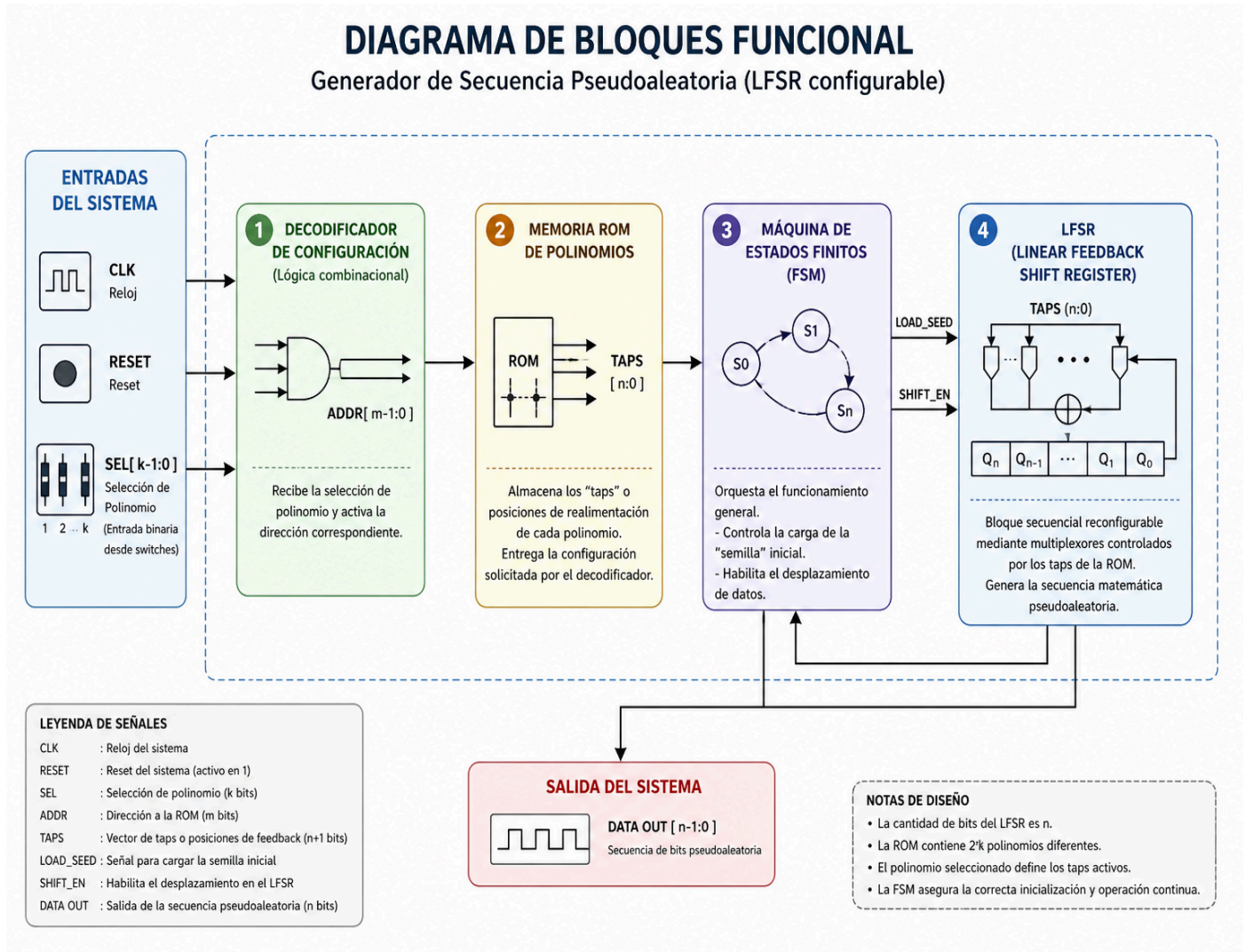


Figura 2: Diagrama ASM

- **Estado IDLE:** $\zeta_{Start} == 1$? (Sí -> Va a READ_ROM) (No -> Se mantiene).
- **Estado READ_ROM:** seed_reg = rom_data. (Pasa a LOAD_PRBS).
- **Estado LOAD_PRBS:** prbs_out = seed_reg. (Pasa a RUN).
- **Estado RUN:** prbs_out = función XOR. $\zeta_{Start} == 0$? (Sí -> Va a IDLE).

4. DIAGRAMA DE BLOQUES



Justificación De Cada Bloque Funcional:

- **Decodificador de Configuración (Lógica Combinacional):** Bloque de entrada indispensable, ya que los interruptores mecánicos externos (SEL[k-1:0]) entregan una selección en código binario que no coincide de forma directa con el mapa físico de memoria. Permite traducir dinámicamente dicha selección del usuario en una dirección binaria concreta (ADDR[m-1:0]), aislando los periféricos de entrada y garantizando la correcta activación del puntero de lectura en la etapa de almacenamiento.

- **Memoria ROM de Polinomios:** Bloque tecnológico modular que rompe el enfoque pasivo de los generadores cableados convencionales al actuar como un repositorio no volátil de coeficientes matemáticos. Almacena de manera fija y ordenada los vectores de realimentación (TAPS[n:0]) correspondientes a cada polinomio de máxima longitud. Su implementación optimiza el uso de recursos lógicos en el hardware, entregando de forma síncrona la configuración solicitada por el decodificador sin necesidad de reestructurar las conexiones físicas del circuito.
- **Máquina de Estados Finitos (FSM):** Orquesta el funcionamiento temporal e inercial de todo el sistema de manera síncrona. Gobierna de forma determinista la secuencia operativa controlando las líneas críticas **LOAD_SEED** y **SHIFT_EN**. Su presencia es fundamental para la seguridad del sistema, ya que asegura un arranque correcto forzando primero la carga de una semilla inicial no nula y habilitando, en el ciclo inmediato, el desplazamiento continuo para evitar el estado prohibido de todo ceros.
- **LFSR (Linear Feedback Shift Register) Reconfigurable:** Bloque secuencial de procesamiento que transforma los parámetros de configuración estáticos provenientes de la ROM en el flujo matemático pseudoaleatorio dinámico. Implementa un arreglo de flip-flops y multiplexores que alteran sus conexiones internas de realimentación en tiempo de ejecución en función de los *taps* seleccionados. Proporciona el comportamiento central del sistema al generar en la salida (DATA\ OUT[n-1:0]) un patrón binario de espectro disperso ideal para la evaluación de canales y pruebas de tasas de error en enlaces de comunicación digital.

5. CÓDIGO HDL COMENTADO

A continuación, se presenta el código desarrollado en Verilog, el cual integra el decodificador, la ROM y la FSM.

```
`timescale 1ns / 1ps

module prbs_system (
    input wire clk, reset, start,
    input wire [1:0] address,
    output reg [7:0] prbs_out,
    output reg valid_out
);
    // Definición de estados para la FSM
    parameter IDLE = 2'b00, READ_ROM = 2'b01, LOAD_PRBS = 2'b10, RUN = 2'b11;
    reg [1:0] current_state;
    reg [3:0] dec_out;
    reg [7:0] rom_data;
    reg [7:0] seed_reg;
```

```

wire feedback;

// Bloque 1: Decodificador de 2 a 4 líneas
always @(*) begin
    case(address)
        2'b00: dec_out = 4'b0001;
        2'b01: dec_out = 4'b0010;
        2'b10: dec_out = 4'b0100;
        2'b11: dec_out = 4'b1000;
        default: dec_out = 4'b0000;
    endcase
end

// Bloque 2: Memoria ROM con 4 semillas de 8 bits
always @(*) begin
    case(dec_out)
        4'b0001: rom_data = 8'b10101011; // Dirección 00
        4'b0010: rom_data = 8'b11110000; // Dirección 01
        4'b0100: rom_data = 8'b00001111; // Dirección 10
        4'b1000: rom_data = 8'b11001100; // Dirección 11
        default: rom_data = 8'b00000001;
    endcase
end

// Ecuación del PRBS (Feedback LFSR) usando compuertas XOR
assign feedback = prbs_out[7] ^ prbs_out[5] ^ prbs_out[4] ^ prbs_out[3];

// Bloque 3: Máquina de Estados (Unidad de Control)
always @(posedge clk or posedge reset) begin
    if (reset) begin
        current_state <= IDLE; // Reseteo del sistema
        prbs_out <= 8'b00000000;
        valid_out <= 1'b0;
    end else begin
        case (current_state)
            IDLE: if (start) current_state <= READ_ROM;
            READ_ROM: begin seed_reg <= rom_data; current_state <= LOAD_PRBS; end
            LOAD_PRBS: begin prbs_out <= seed_reg; current_state <= RUN; end
            RUN: begin
                valid_out <= 1'b1;
                prbs_out <= {prbs_out[6:0], feedback}; // Desplazamiento
                if (!start) current_state <= IDLE;
            end
        endcase
    end
end
endmodule

```

6. PLAN DE SIMULACIÓN CON CASOS DE PRUEBA

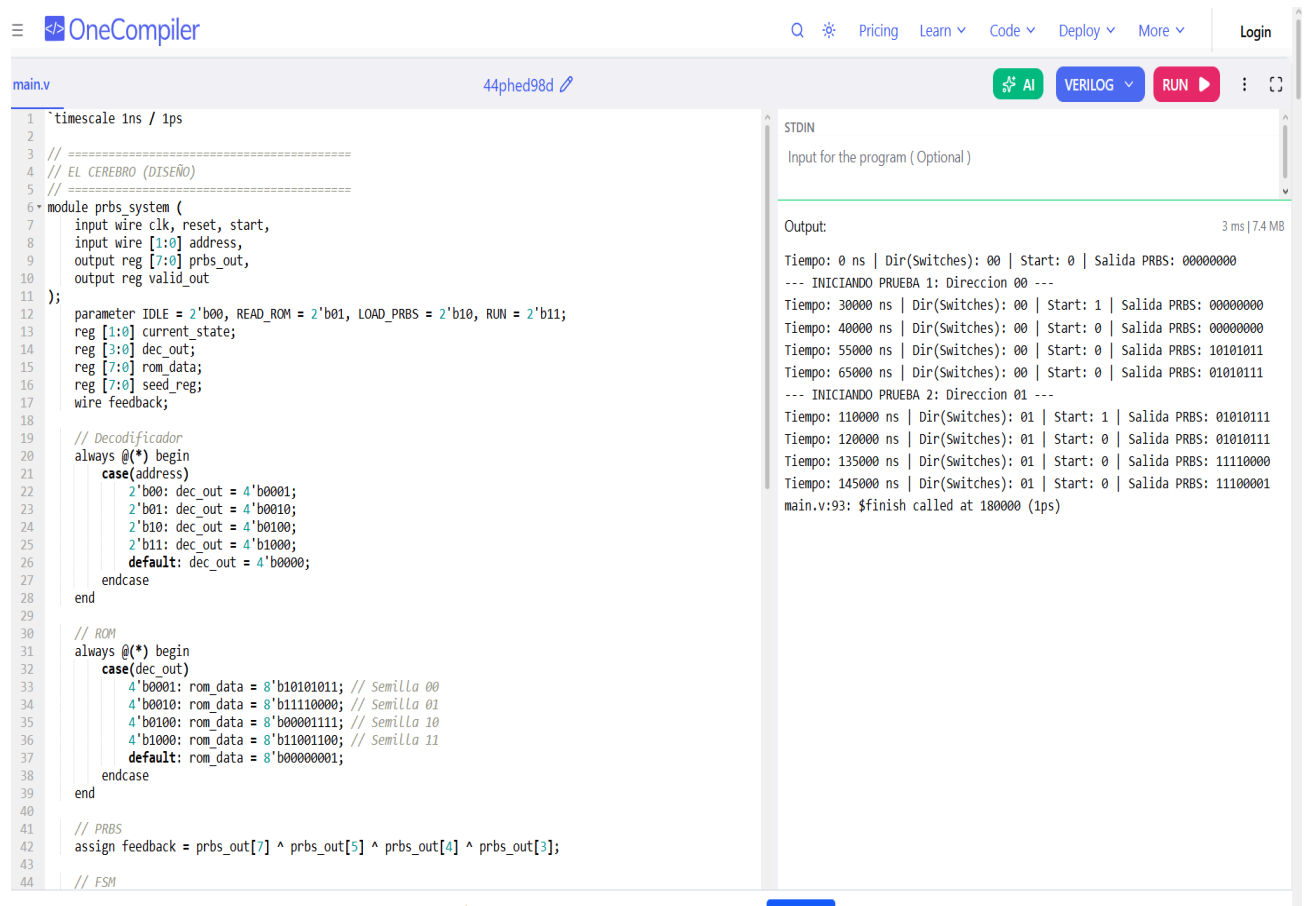
Para verificar el correcto funcionamiento, se diseñó un *Testbench* que ejecuta los siguientes casos de prueba:

Caso de Prueba 1: Se aplica la dirección **00** en los interruptores y se activa la señal **start**. El sistema debe acceder a la ROM, extraer la semilla correspondiente (**10101011**) y comenzar la generación pseudoaleatoria.

Caso de Prueba 2: Se cambia la dirección a **01** y se reactiva el **start**. El sistema debe actualizarse y cargar la nueva semilla (**11110000**).

7. RESULTADOS Y CAPTURAS DE PANTALLA COMENTADAS

La simulación funcional se realizó imprimiendo los registros de salida mediante la consola (STDOUT).



```
1 `timescale 1ns / 1ps
2
3 // =====
4 // EL CEREBRO (DISEÑO)
5 // =====
6 module prbs_system (
7     input wire clk, reset, start,
8     input wire [1:0] address,
9     output reg [7:0] prbs_out,
10    output reg valid_out
11);
12    parameter IDLE = 2'b00, READ_ROM = 2'b01, LOAD_PRBS = 2'b10, RUN = 2'b11;
13    reg [1:0] current_state;
14    reg [3:0] dec_out;
15    reg [7:0] rom_data;
16    reg [7:0] seed_reg;
17    wire feedback;
18
19    // Decodificador
20    always @(*) begin
21        case(address)
22            2'b00: dec_out = 4'b0001;
23            2'b01: dec_out = 4'b0010;
24            2'b10: dec_out = 4'b0100;
25            2'b11: dec_out = 4'b1000;
26            default: dec_out = 4'b0000;
27        endcase
28    end
29
30    // ROM
31    always @(*) begin
32        case(dec_out)
33            4'b0001: rom_data = 8'b10101011; // Semilla 00
34            4'b0010: rom_data = 8'b11110000; // Semilla 01
35            4'b0100: rom_data = 8'b00001111; // Semilla 10
36            4'b1000: rom_data = 8'b11001100; // Semilla 11
37            default: rom_data = 8'b00000001;
38        endcase
39    end
40
41    // PRBS
42    assign feedback = prbs_out[7] ^ prbs_out[5] ^ prbs_out[4] ^ prbs_out[3];
43
44    // FSM
```

STDOUT

Input for the program (Optional)

Output: 3 ms | 7.4 MB

Tiempo: 0 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 00000000
--- INICIANDO PRUEBA 1: Direccion 00 ---
Tiempo: 30000 ns | Dir(Switches): 00 | Start: 1 | Salida PRBS: 00000000
Tiempo: 40000 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 00000000
Tiempo: 55000 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 10101011
Tiempo: 65000 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 01010111
--- INICIANDO PRUEBA 2: Direccion 01 ---
Tiempo: 110000 ns | Dir(Switches): 01 | Start: 1 | Salida PRBS: 01010111
Tiempo: 120000 ns | Dir(Switches): 01 | Start: 0 | Salida PRBS: 01010111
Tiempo: 135000 ns | Dir(Switches): 01 | Start: 0 | Salida PRBS: 11110000
Tiempo: 145000 ns | Dir(Switches): 01 | Start: 0 | Salida PRBS: 11100001
main.v:93: \$finish called at 180000 (1ps)

Output:

3 ms | 7.4 MB

```
Tiempo: 0 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 00000000
--- INICIANDO PRUEBA 1: Direccion 00 ---
Tiempo: 30000 ns | Dir(Switches): 00 | Start: 1 | Salida PRBS: 00000000
Tiempo: 40000 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 00000000
Tiempo: 55000 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 10101011
Tiempo: 65000 ns | Dir(Switches): 00 | Start: 0 | Salida PRBS: 01010111
--- INICIANDO PRUEBA 2: Direccion 01 ---
Tiempo: 110000 ns | Dir(Switches): 01 | Start: 1 | Salida PRBS: 01010111
Tiempo: 120000 ns | Dir(Switches): 01 | Start: 0 | Salida PRBS: 01010111
Tiempo: 135000 ns | Dir(Switches): 01 | Start: 0 | Salida PRBS: 11110000
Tiempo: 145000 ns | Dir(Switches): 01 | Start: 0 | Salida PRBS: 11100001
main.v:93: $finish called at 180000 (1ps)
```

Análisis de la simulación: Como se observa en la captura de pantalla:

1. **En el tiempo 55000 ns:** Al evaluar la dirección **00**, el sistema carga exitosamente la semilla correcta **10101011** directamente desde la memoria ROM.
2. **En el tiempo 65000 ns:** Se evidencia que el estado RUN de la máquina de estados funciona, ya que el PRBS comienza a iterar y el valor cambia a **01010111**.
3. **En el tiempo 135000 ns:** Al aplicar el Caso de Prueba 2 (Dirección **01**), el decodificador actúa correctamente leyendo la nueva posición de la ROM e insertando la semilla **11110000** en el sistema.

8. CONCLUSIONES

Se logró diseñar e implementar exitosamente el generador PRBS con semilla configurable. La simulación demostró que la máquina de estados (FSM) coordina de forma efectiva el decodificador de direcciones y la lectura de la memoria ROM antes de habilitar el generador de números aleatorios, cumpliendo con todos los requisitos funcionales del proyecto.

9. REFERENCIAS

B. Sklar, *Digital Communications: Fundamentals and Applications*, 2da ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2001.

<https://books.google.com/books?id=c111GQAACAAJ>

P. P. Chu, *FPGA Prototyping by VHDL Examples: Altera DE-Series Board Edition*. Hoboken, NJ, USA: John Wiley & Sons, 2011.

<https://onlinelibrary.wiley.com/doi/book/10.1002/9781118080644>

S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 3ra ed. Nueva York, NY, USA: McGraw-Hill, 2008.

<https://books.google.com/books?id=2D9QAAAAMAAJ>

S. S. S. Reddy, P. R. Kumar, y T. V. K. HanumanthaRao, "Design and implementation of PRBS generator using LFSR," en *Proc. IEEE Int. Conf. Commun. Signal Process. (ICCSP)*, Melmaruvathur, India, 2015, pp. 1157-1160.**

<https://ieeexplore.ieee.org/document/7322687>